Graphs, Matrices and Algorithms

29/~~Sep~~/25

26/Oct/26

I'm SUBRAHMANYAM KALYANASUNDARAM
Dept of CSE, IIT HYDERABAD.
EMAIL: SUBRUK@CSE.IITH.AC.IN

I'll be discussing some basics of graph algorithms over the course of 4 lectures.

Algorithm: A well defined computational procedure that takes some values as input and produces some value or values as output.

Could be decision problem (does this graph have a triangle?) or function computation (given a graph, color graph using the smallest no. of colors).

Lots of applications : Graphs can be used to model different kinds of relations, networks and it could be useful to compute the parameters / properties of such graphs. (Is a graph connected ? Does a graph have a specific subgraph ? Find a shortest path from X to Y.)

How do you measure the performance of algorithms ?

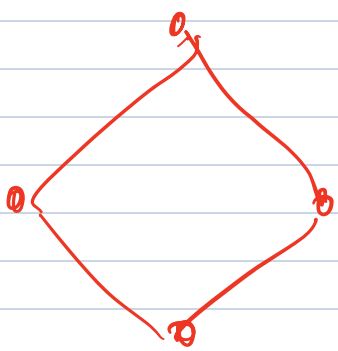
→ An algorithm is a description of a process.

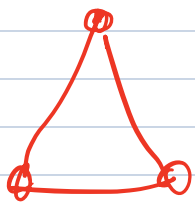
→ The most common resources of computation are time and space.

→ Time will vary with hardware (processor speed, for instance) and with input size.

⑥ $n^2/\log n + n^{1.99}$ is $\Theta(n^2/\log n)$

$\frac{n^2}{n^{0.01}}$



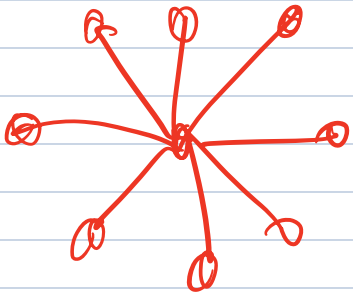


3-cycle



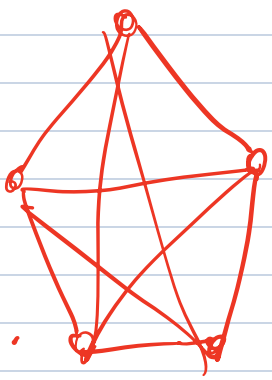
Path on 4 vertices

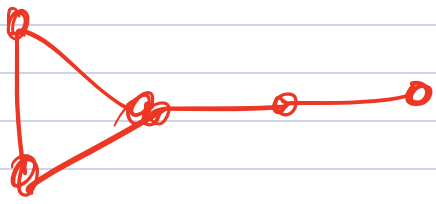
Star



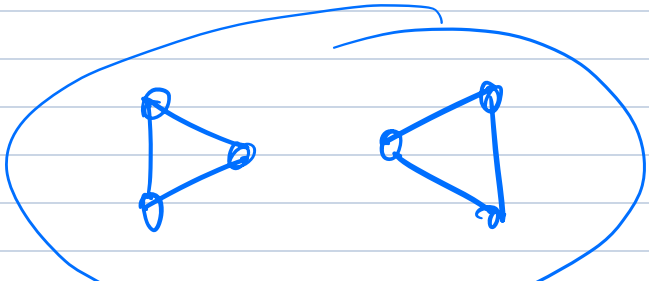
4-cycle

Complete graph on 5 vertices.





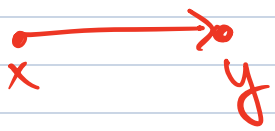
Disconnected



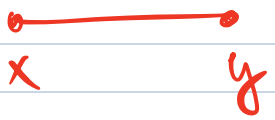
Graphs

We have $G = (V, E)$ a graph, where V is the set of vertices and E is the set of edges.

→ Directed graph: The edges are directed and are ordered pairs (x, y) where $x, y \in V$

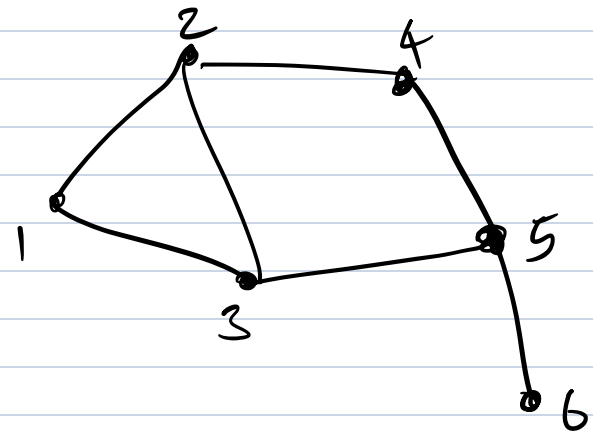


→ Undirected graph: The edges are unordered pairs $\{x, y\}$ where $x, y \in V$



The application will dictate whether the graph is directed or undirected.

How do we represent graphs?



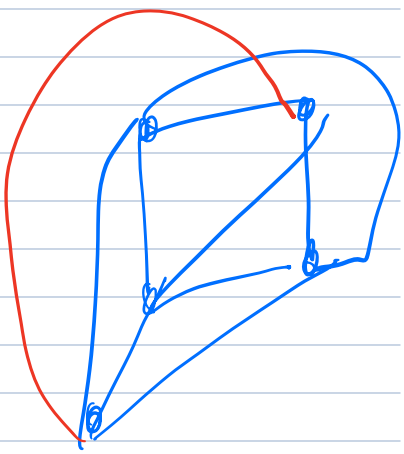
Adjacency matrix

We have an $n \times n$ matrix where $n = |V|$.

The $(i,j)^{th}$ entry is $1 \iff (i,j) \in E$ there is an edge from i to j

Else the $(i,j)^{th}$ entry is 0

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}$$



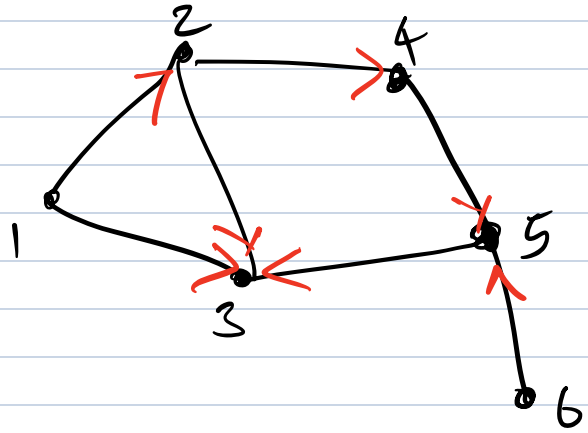
Adj matrix needs $O(n^2)$ space.

But we can immediately say if (i,j) is an edge. → Done in $O(1)$ time.

constant

The matrix A is symmetric if and only if the graph is undirected.

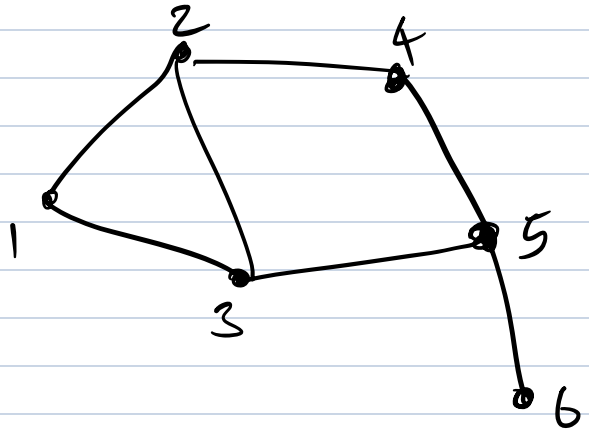
What is the adjacency matrix of this directed graph?



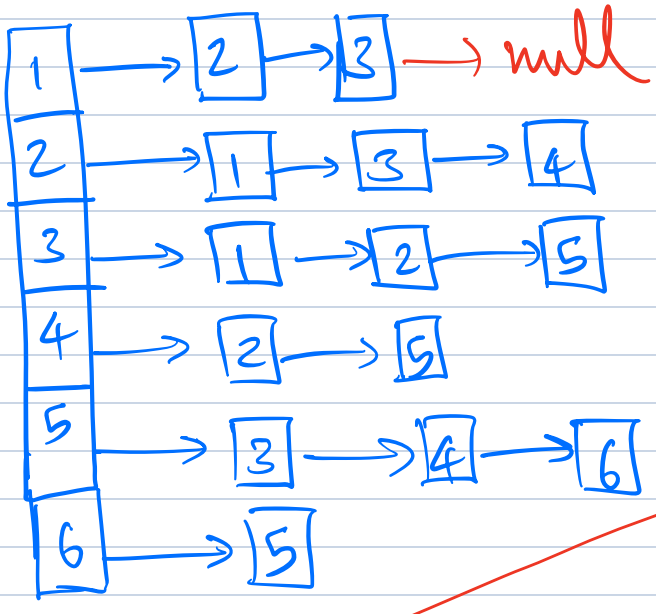
$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{array} \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$(i,j)^{th}$ entry is 1 if there is an edge from vertex i to vertex j

Adjacency list



This is a different way to represent a graph. Here we have $n = |V|$ linked lists, one for each vertex. Each one contains the list of (out)-neighbors of a vertex.



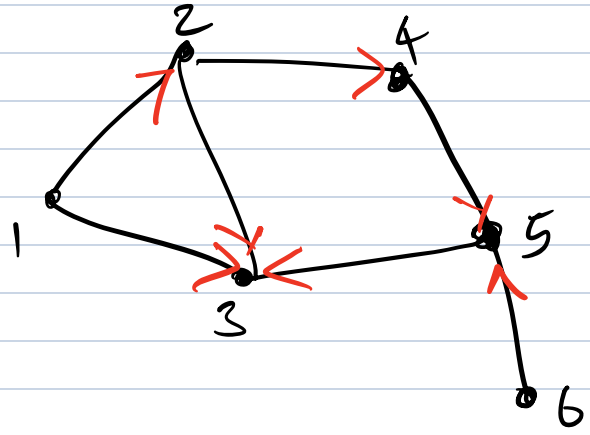
Max no. of edges in an n vertex graph = $\binom{n}{2}$ = $\frac{n(n-1)}{2}$

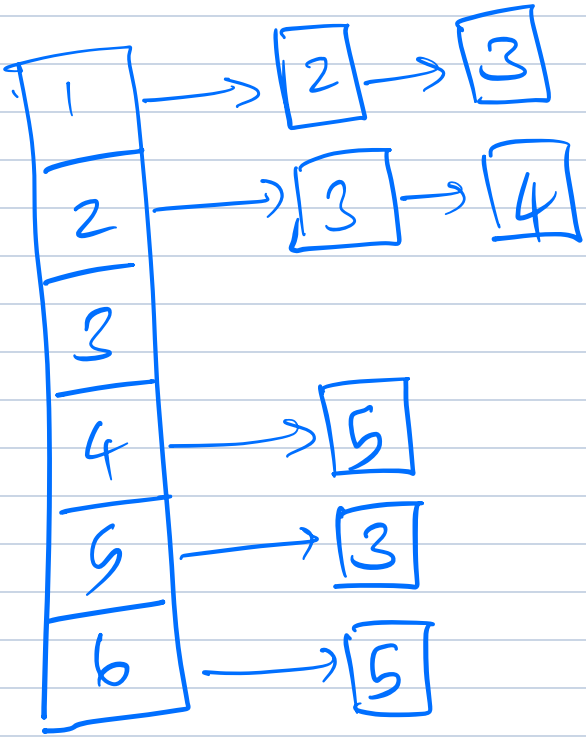
This takes $O(|E|)$ space. (In an undirected graph, each edge appears two times).

But checking if (i, j) is an edge may not be immediate. It can take $O(|V|)$ time.

Can take up to $d(v)$ time for vertex v.

What is the adjacency list of this directed graph?





Which representation is better?

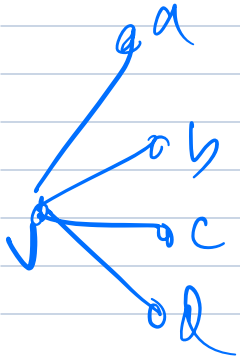
Adjacency matrix takes $O(|V|^2) = O(n^2)$ space. But can immediately check adjacencies.

$O(1)$ time

Adjacency list takes only $O(|E|)$ space but can take up to $O(|V|)$ time to check adjacencies.

↳ degree(v) where v is the endpoint of the query

If $|E|$ is $O(|V|^2)$ then adjacency matrix may be better as adj. list also takes $O(|E|) = O(|V|^2)$ space.



If we have a sparse graph, i.e., where $O(|E|) = O(|V|)$ or $O(|V| \log |V|)$

(something smaller than $O(|V|^2)$)

then it may help to use adj. list.

Adjacency list also helps in case of applications where we want to go through all the neighbors of a given vertex, u.

Graph Exploration

We first consider one of the simplest questions one can ask about a graph.

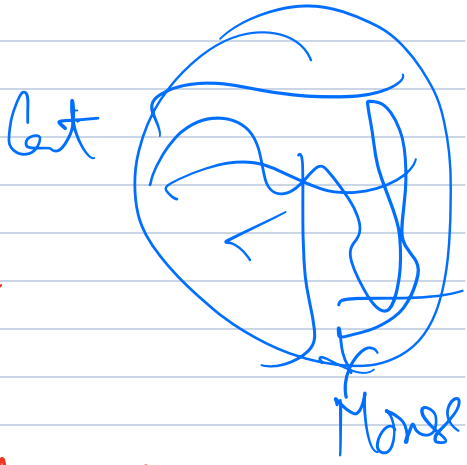
Q: If we start from a vertex $s \in V$, which other vertices are reachable?

Suppose you are in a maze, how can you find which are the reachable vertices?

→ Rope and Chalk

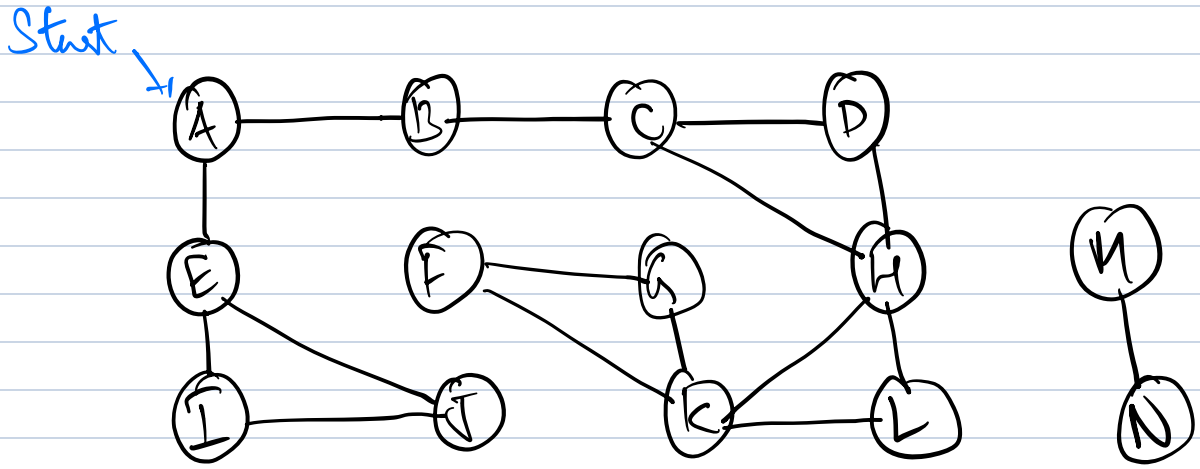


to return to starting point

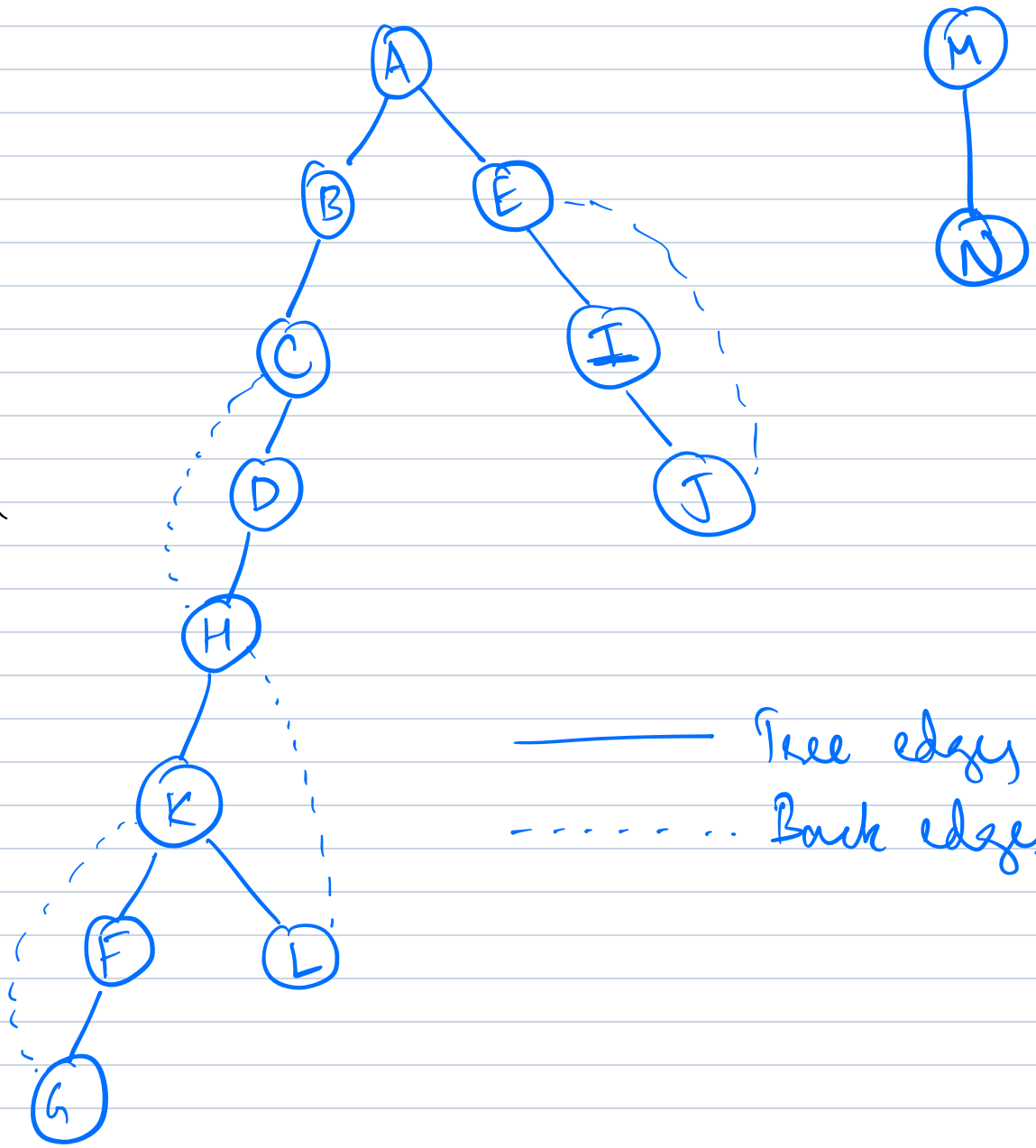


to mark visited nodes.

Can we do the same with a computer?



Depth First Search (DFS)



Depth First Search (DFS)

(We'll later see Breadth First Search)

```
DFS(G)
For all v ∈ V                      } O(|V|)
    visited(v) = false

For all v ∈ V
    If visited(v) = false, Explore(v)
```

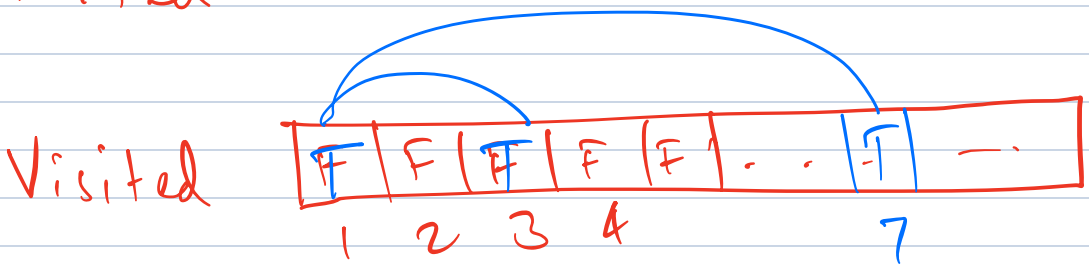
Explore(w) → Discovers all the vertices that are reachable from w.

```
visited(w) = True
Previsit(w)
For each edge (w,z) ∈ E            ← Outgoing edge from w
    If visited(z) = false, Explore(z)
Postvisit(w)
```

This will be defined based on need.

can take deg(w) time

To execute this, we need an array visited



and a representation of the graph.

Explore (w) determines all the vertices reachable from w that have not already been explored.

Correctness: We can prove the following using induction on k.

> In Explore(w) all vertices that are reachable in $\leq k$ steps from w get visited.

Running time analysis:

Initializing all vertices as not visited : $O(|V|)$

Inside explore(w), (assume previsit, to be $O(1)$ post visit

setting visited to True : $O(1)$.

Each edge (x,y) in G is checked exactly twice,

nee during explore(x) and nee during explore(y). This takes $O(|E|)$ time, nee the course of the DFS.

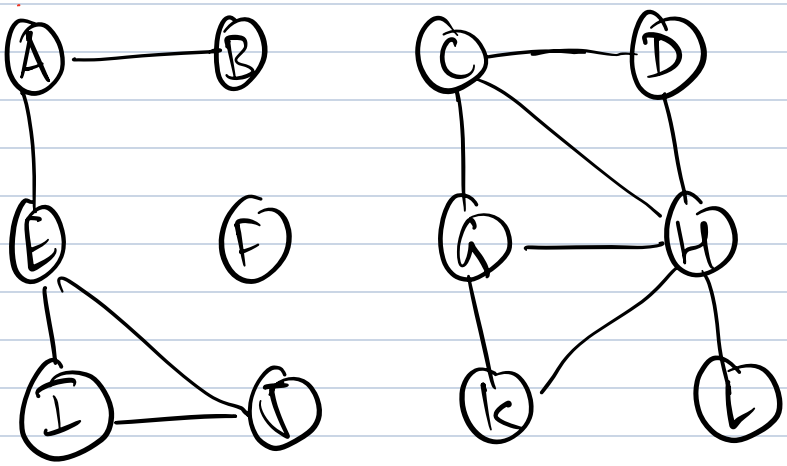
Assuming graph G is given in adj. list format, this takes $O(|V|+|E|)$ time.

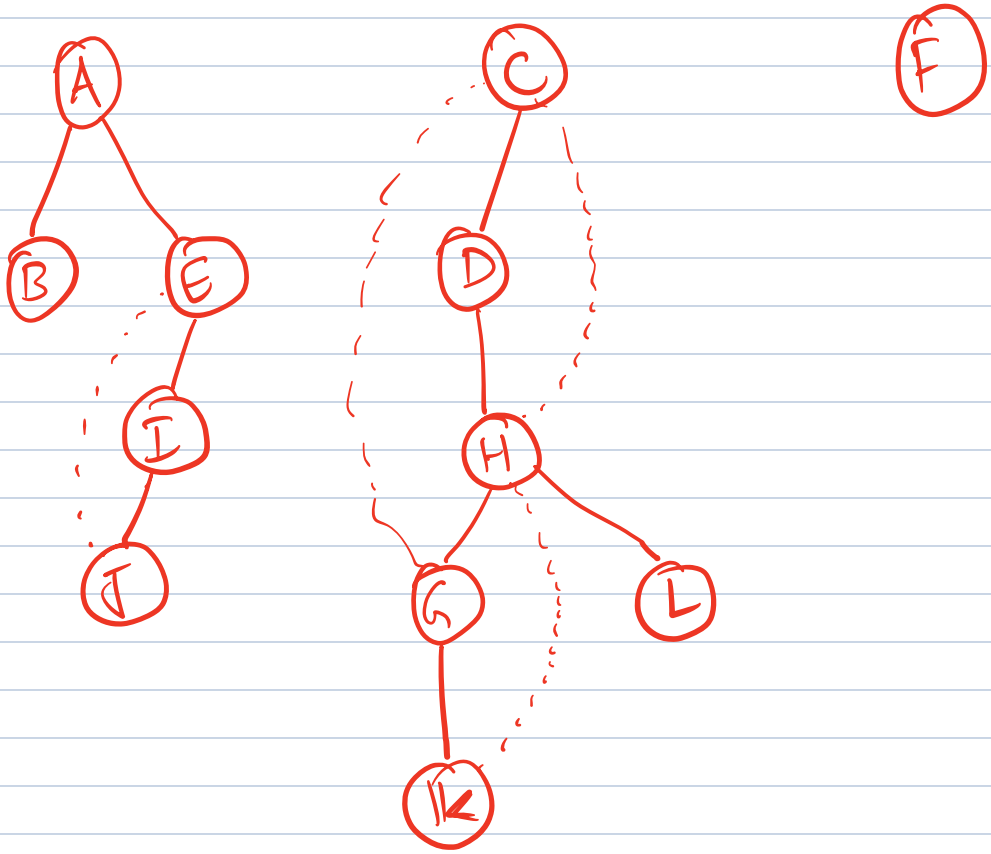
Why is this needed?

If G is given as adj matrix, we will need $O(|V|^2)$ time

This is because we will have to go through the entire row corresponding to vertex v to do Explore(v). Across all vertices v, this results in $O(|V|^2) = O(n^2)$ time.

One application: Want to identify which vertex belongs to which connected component.





ccnum

1	1	2	2	1	3	2	2	1	1	2	2

A B C D E F G H I J K L

We modify DFS as follows.

DFS(G)

For all $v \in V$

    visited(v) = false
    ccnum(v) = Empty

cc = 0

For all $v \in V$

    If visited(v) = false

        cc = cc+1
        Explore(v)

Explore(w)

Visited(w) = True

ccnum(w) = cc

For each edge $(w, z) \in E$

    If visited(z) = false, Explore(z)

The running time remains $O(|V|+|E|)$

Previsit and Postvisit orderings

Initially, we set a clock to one. Then we increment it when we begin exploring a vertex and completing the exploration.

Explore(w)

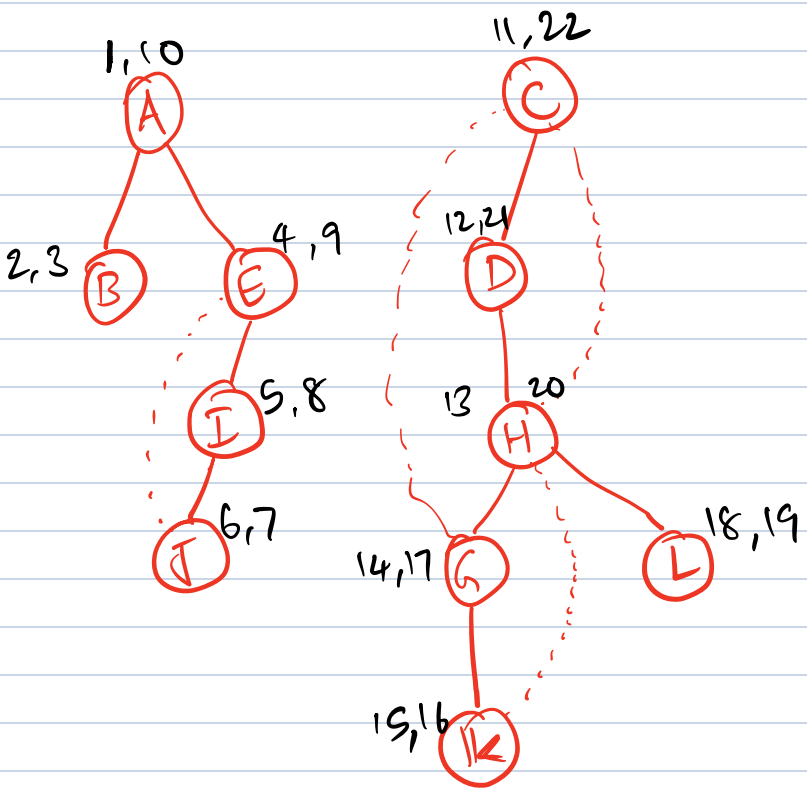
Visited(w) = True

Previsit(w): Pre(w) = clock; clock = clock + 1

for each edge (w, z) ∈ E

If visited(z) = false, Explore(z)

Postvisit(w): Post(w) = clock, clock = clock + 1



[Pre(v), Post(v)]

Observation: For any two $u, v \in V$, $[Pre(u), Post(u)]$ and $[Pre(v), Post(v)]$ are either disjoint or one is contained in the other.

Suppose WLOG, explore(v) is called after explore(u)

Either explore(v) is called as a descendant of explore(u), or after explore(u) is completed and exited.

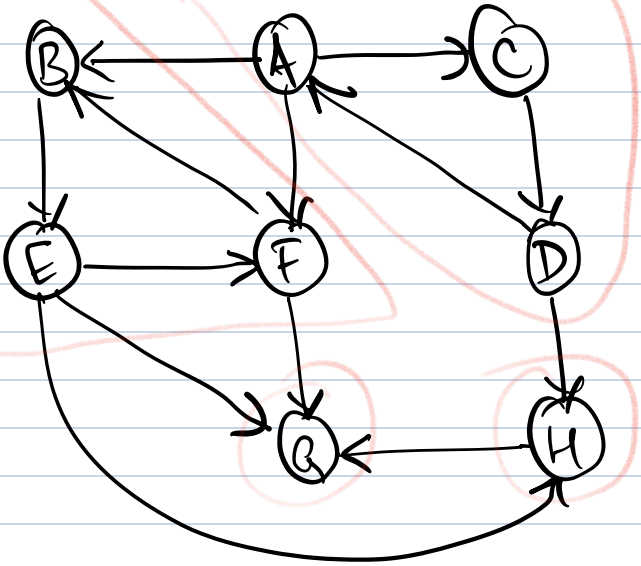
(In other words, is u still in the recursion stack while explore(v) is called?)

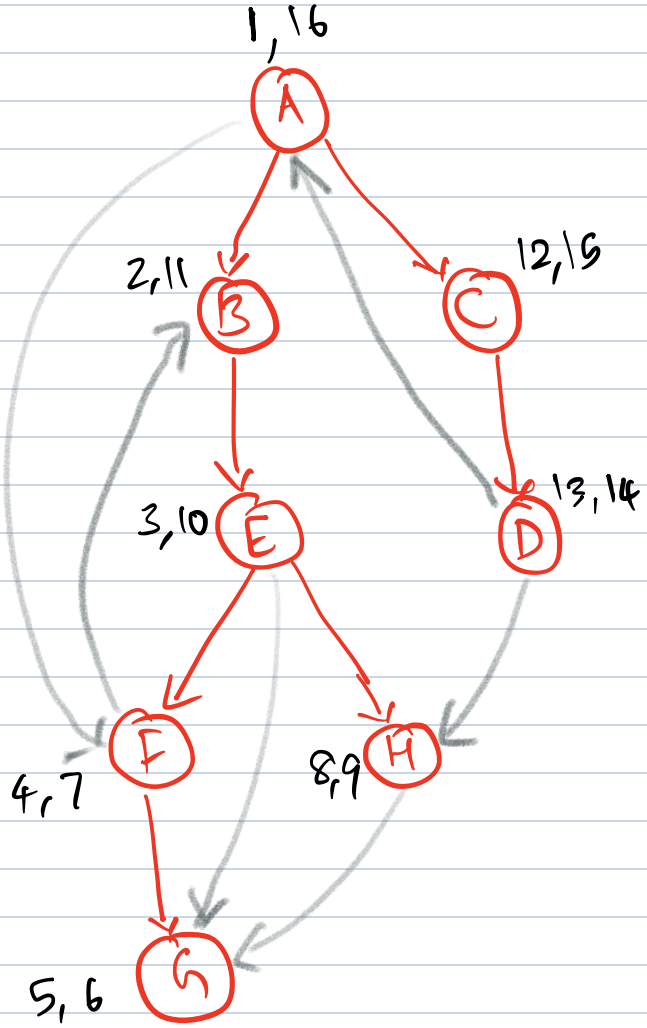
In the former case, $[Pre(v), Post(v)]$ will be contained in $[Pre(u), Post(u)]$.

In the latter case, they will be disjoint.

DFS in Directed Graphs.

In undirected graphs, we just had DFS edges and back edges. Here we will have other type of edges.



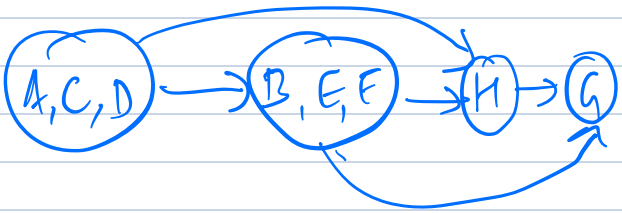


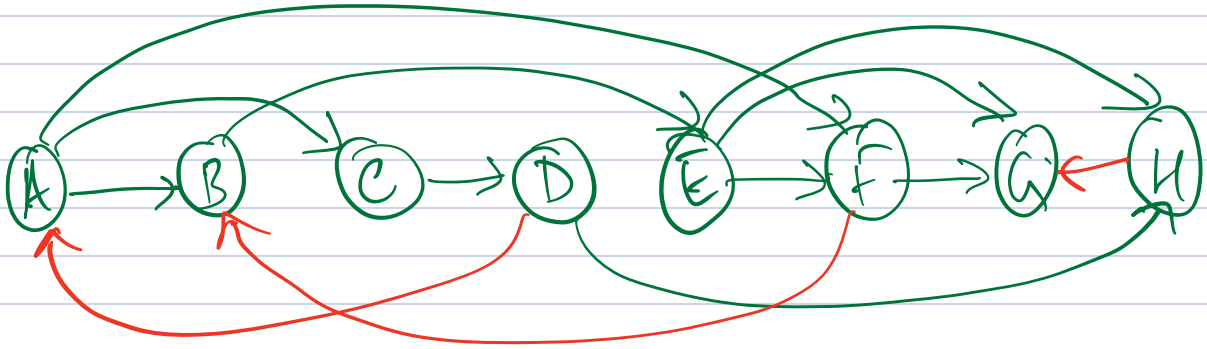
Types of other edges

1. Forward edges: Ancestor to non child descendant
2. Back edges: Descendant to ancestor
3. Cross edges: To neither descendant nor ancestor.

→ Tree edges

→ Other edges.





Question: Is there an ordering of the vertices of this graph so that all the edges go from left to right? → Such an ordering is called topological sort.

No. It's not possible for this graph, since it contains a cycle. So we will need some edge that goes from right to left.

What if a given graph G, does not have a cycle?

↳ Does G necessarily have a topological sort?

How can we identify the edge types?

① For edge (u, v), it is a forward edge or tree edge if ..

$pre(u) < pre(v) < post(v) < post(u)$

v is in u's subtree

② Edge (u, v) is a back edge if

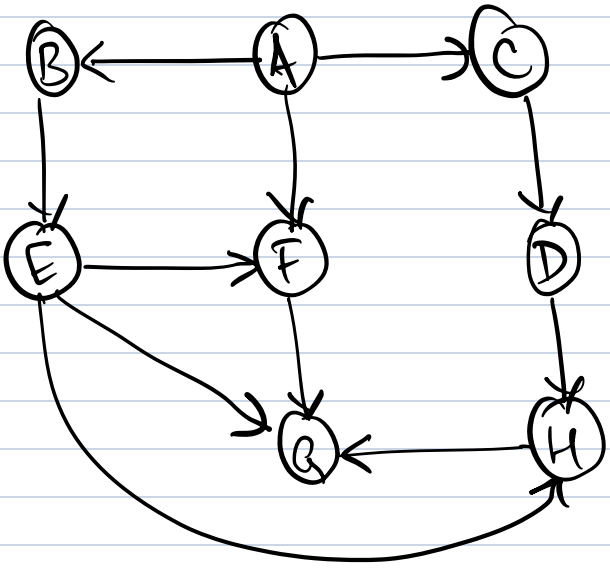
$pre(v) < pre(u) < post(u) < post(v)$

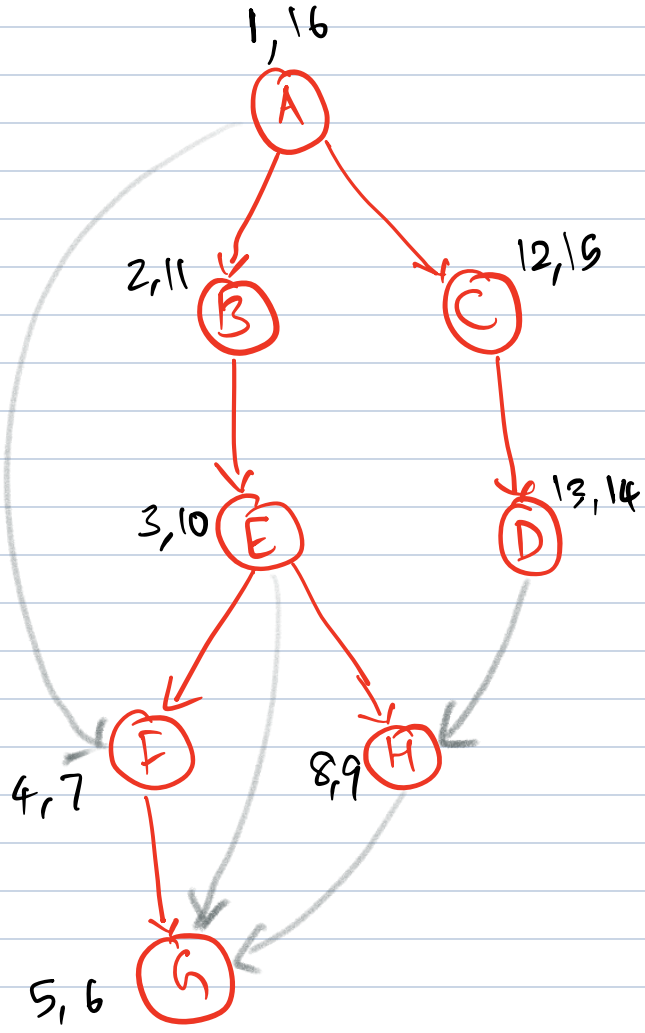
u is in v's subtree.

③ Edge (u, v) is a cross edge if

$pre(v) < post(v) < pre(u) < post(u)$.

DFS exits from v before it explores u.

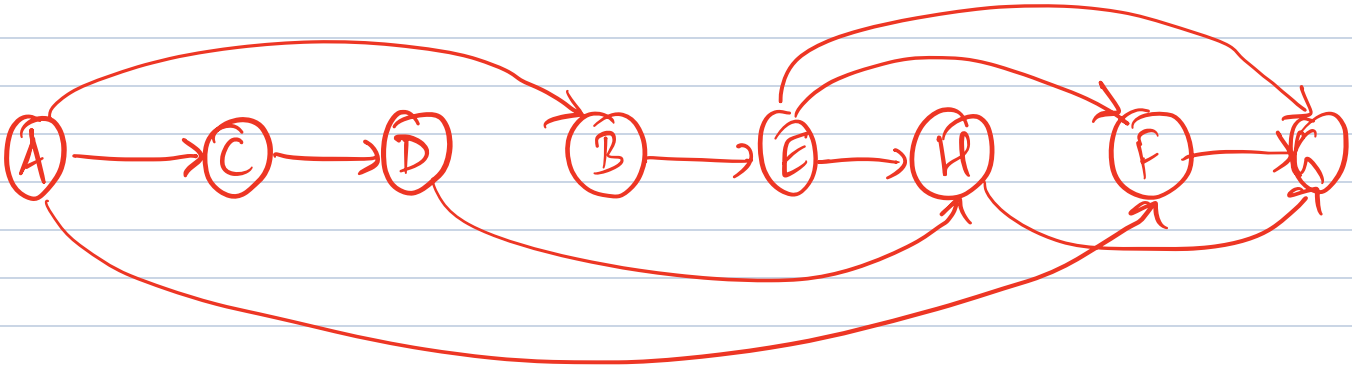




no back edges in DAG!

We already saw that for all edges (u, v) in a DAG, post $(v) <$ post (u).

So we can sort in the descending order of post(·) numbers!



Note: This need not be unique. There may be other such orderings.

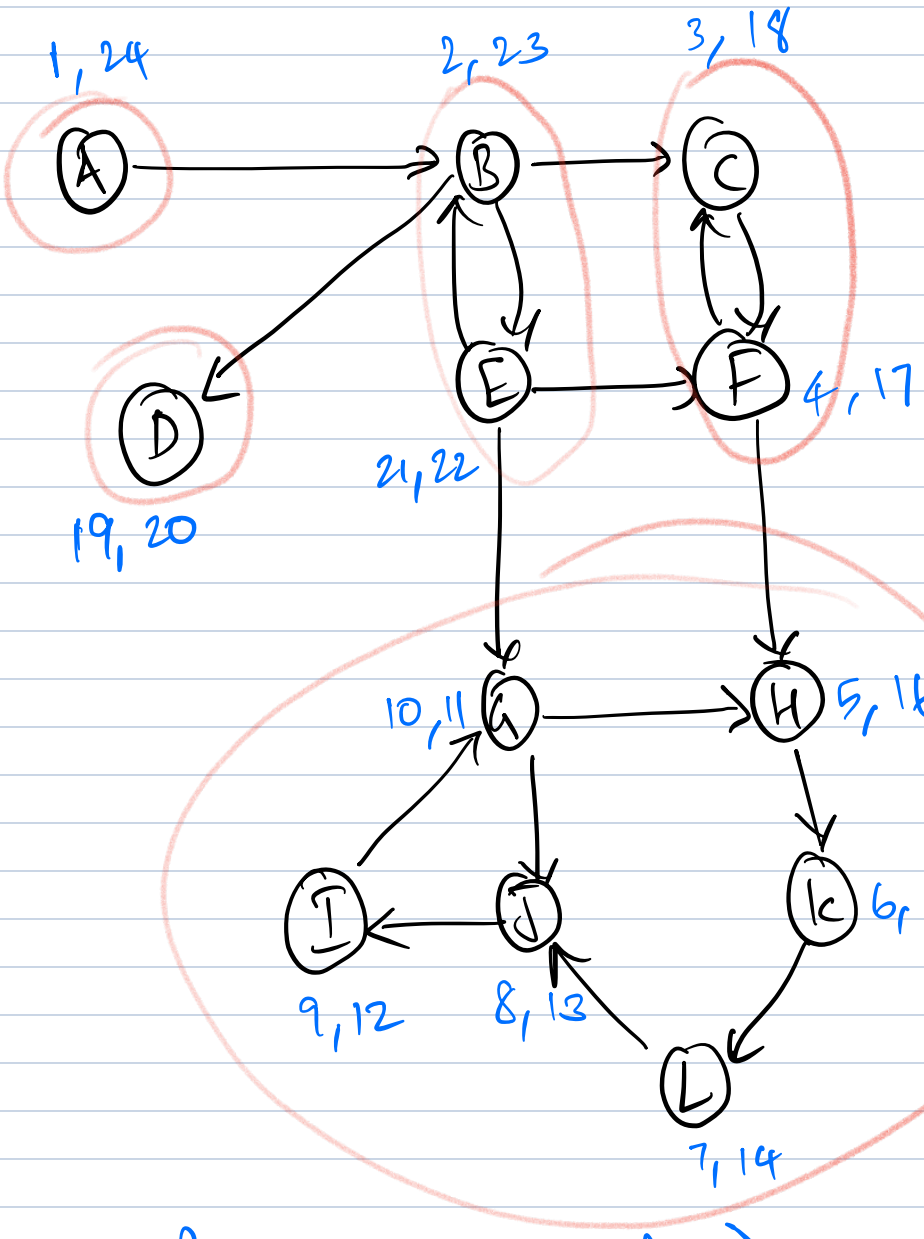
Strongly Connected Components

How can we generalize connected components for directed graphs? It is not as easy or straightforward as undirected graphs.

We say a set $S \subseteq V$ is **strongly connected** if for any $u, v \in S$, there is a directed path from u to v AND there is a directed path from v to u.

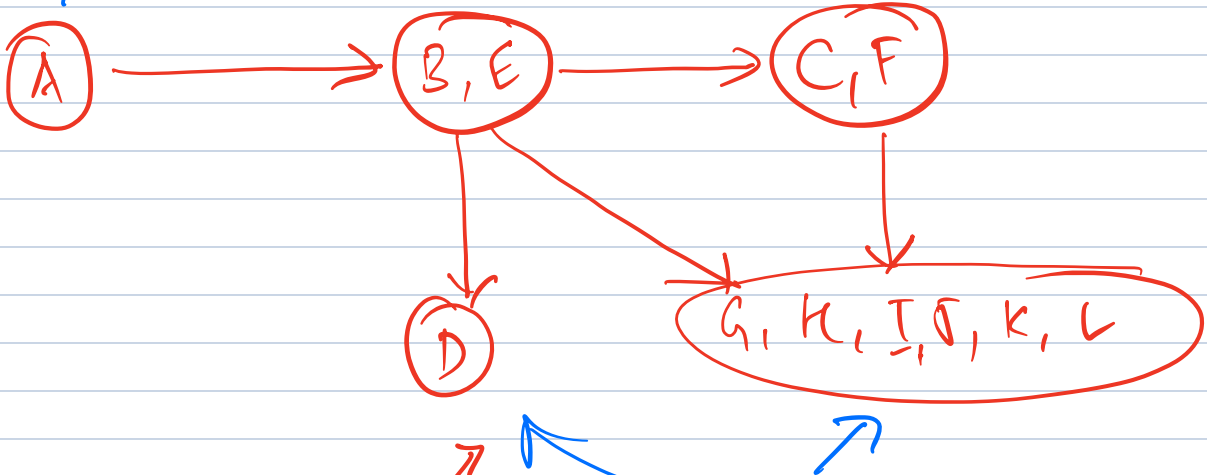
Using this, we can partition V into SCC's (strongly conn. components).

Property: Every directed graph is a DAG of its strongly connected components.



WHY?

Source (no incoming edges)



Meta Graph.

Sinks (no outgoing edges)

Algorithm for SCC

1. Run DFS on G^R.

Can be done in parallel to step 1.

2. Order the vertices in G in the decreasing order of the post numbers in step 1.

3. Run DFS on G where we use the order of the vertices obtained in step 2.

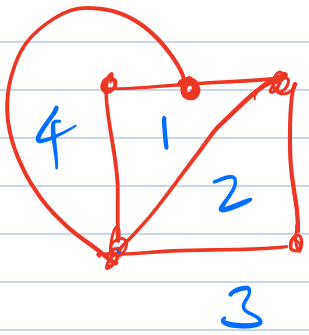
The connected components algorithm applied on this ordering will yield the SCC's.

$2 \times DFS \Rightarrow O(|V| + |E|)$ time.

References: ① Introduction to Algorithms by Cormen, Leiserson, Rivest, Stein

② Algorithms by Dasgupta, Papadimitriou and Vazirani.

Euler's formula



$$V + F = E + 2$$

↓ vertices ↘ faces ↘ edges.

Each face has at least 3 boundary edges.

$$E \geq \tfrac{3}{2} F \Rightarrow F \leq \tfrac{2}{3} E$$

$$V = E - F + 2 \geq \frac{E}{3} + 2$$

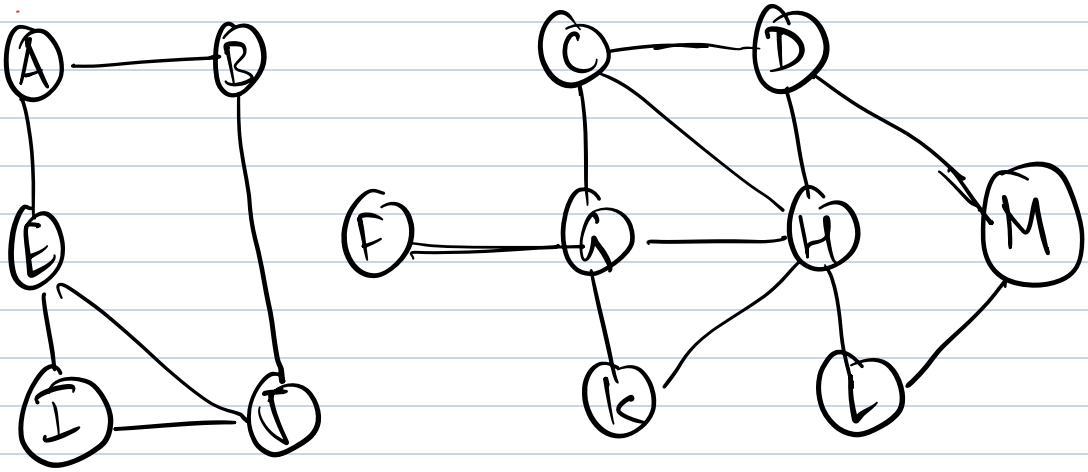
$$E \leq 3V - 2$$

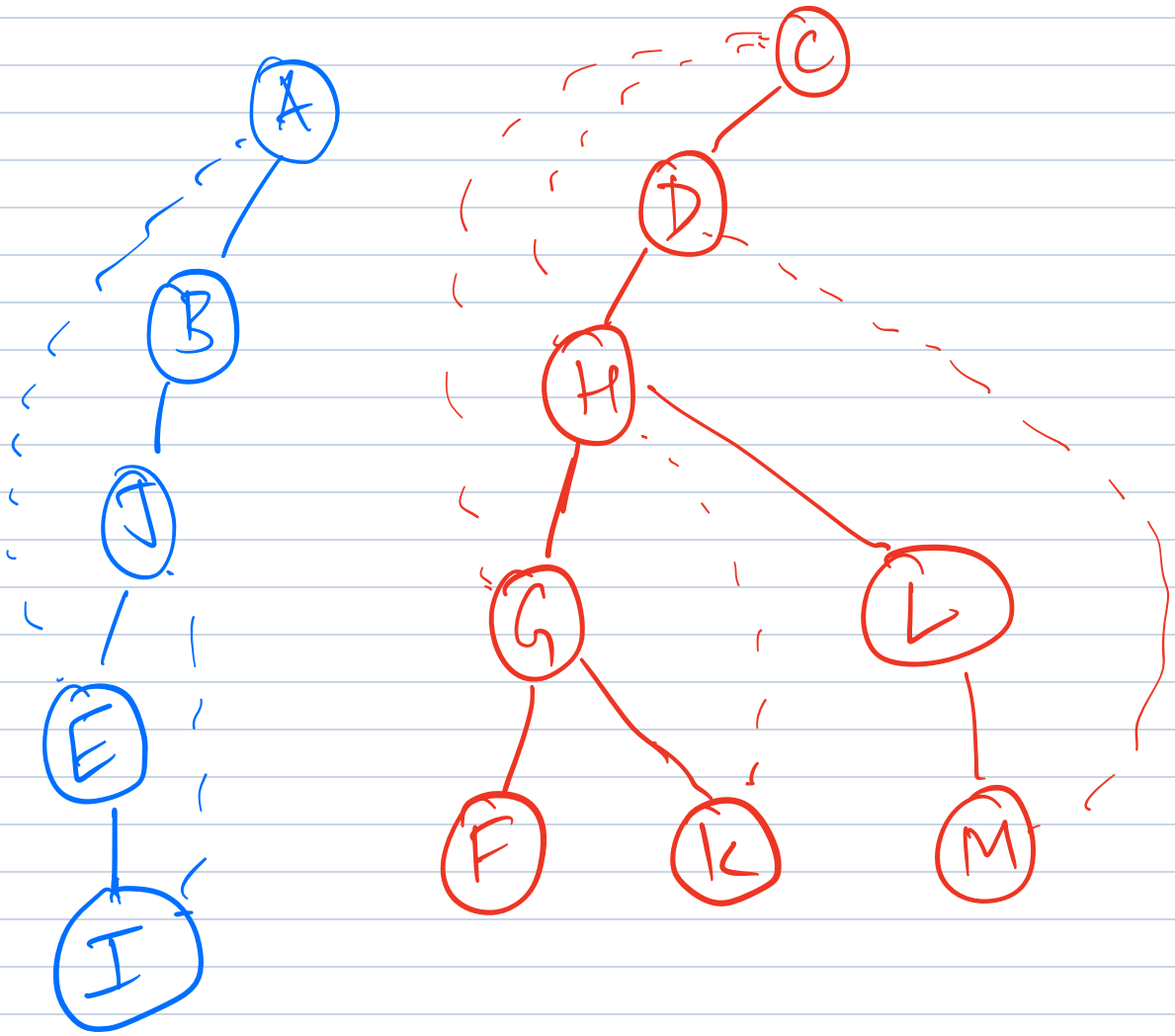
Each edge contributes 2 end points

$$\sum_{v \in V} d(v) = 2|E| \leq 6V - 4$$

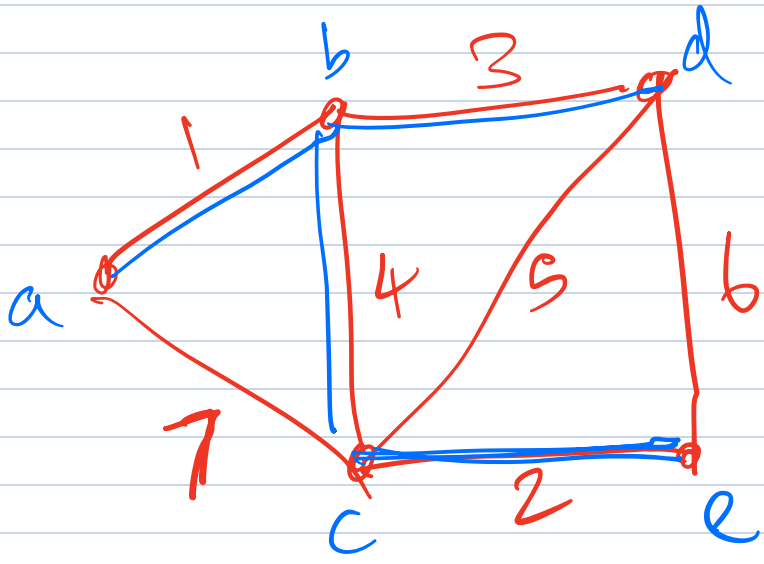
$d(v)$: degree

$\Rightarrow$ There is a vertex with degree ≤ 6.


A
B
E
I
J
C
D
F
G
H
M
K
L

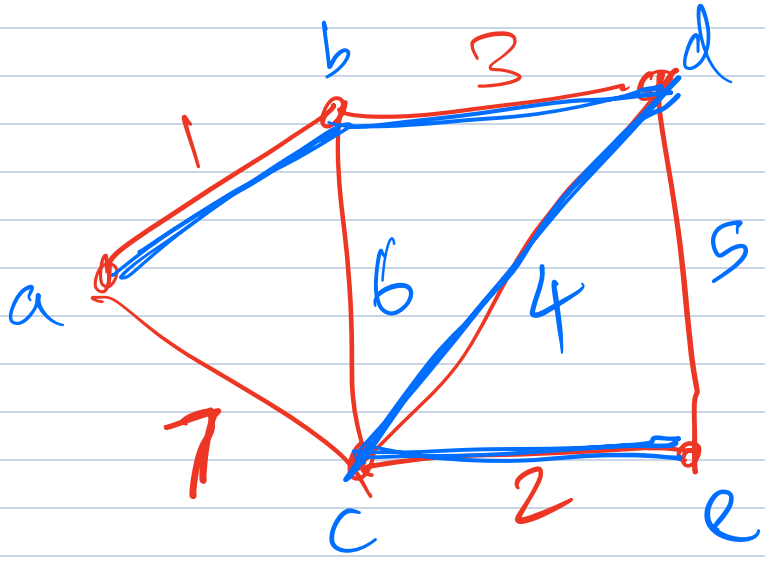

A
B
J
E
I
C
D
H
G
L
F
K
M

Spanning trees (02/Nov/2026)

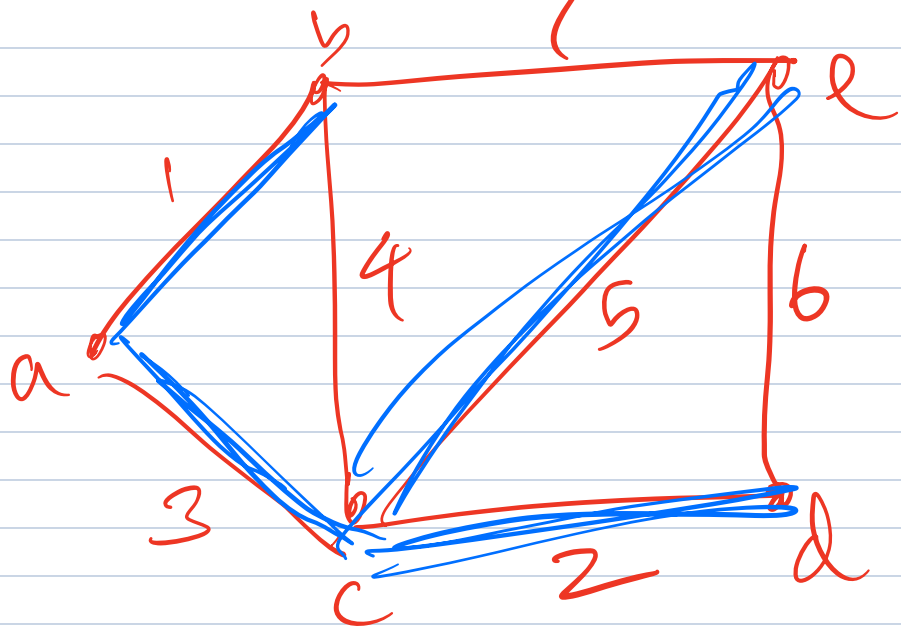


10

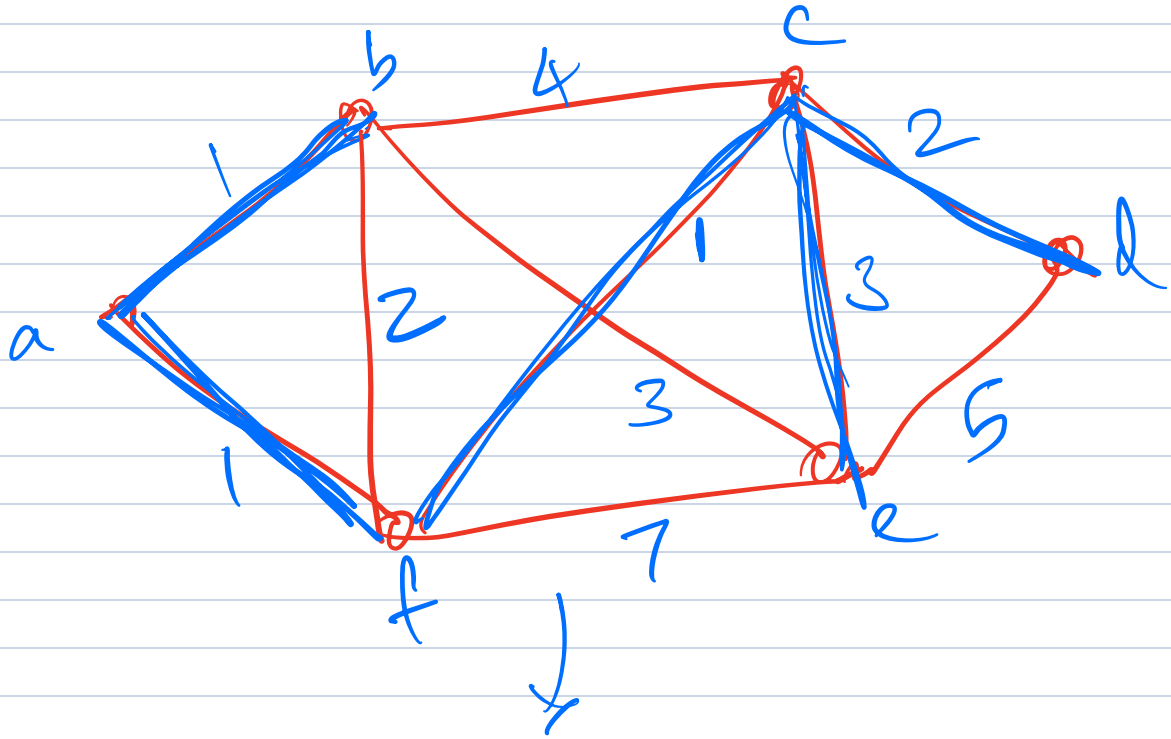
MST:



wt
10



wt 11



This graph has min spanning tree of weight = 8.

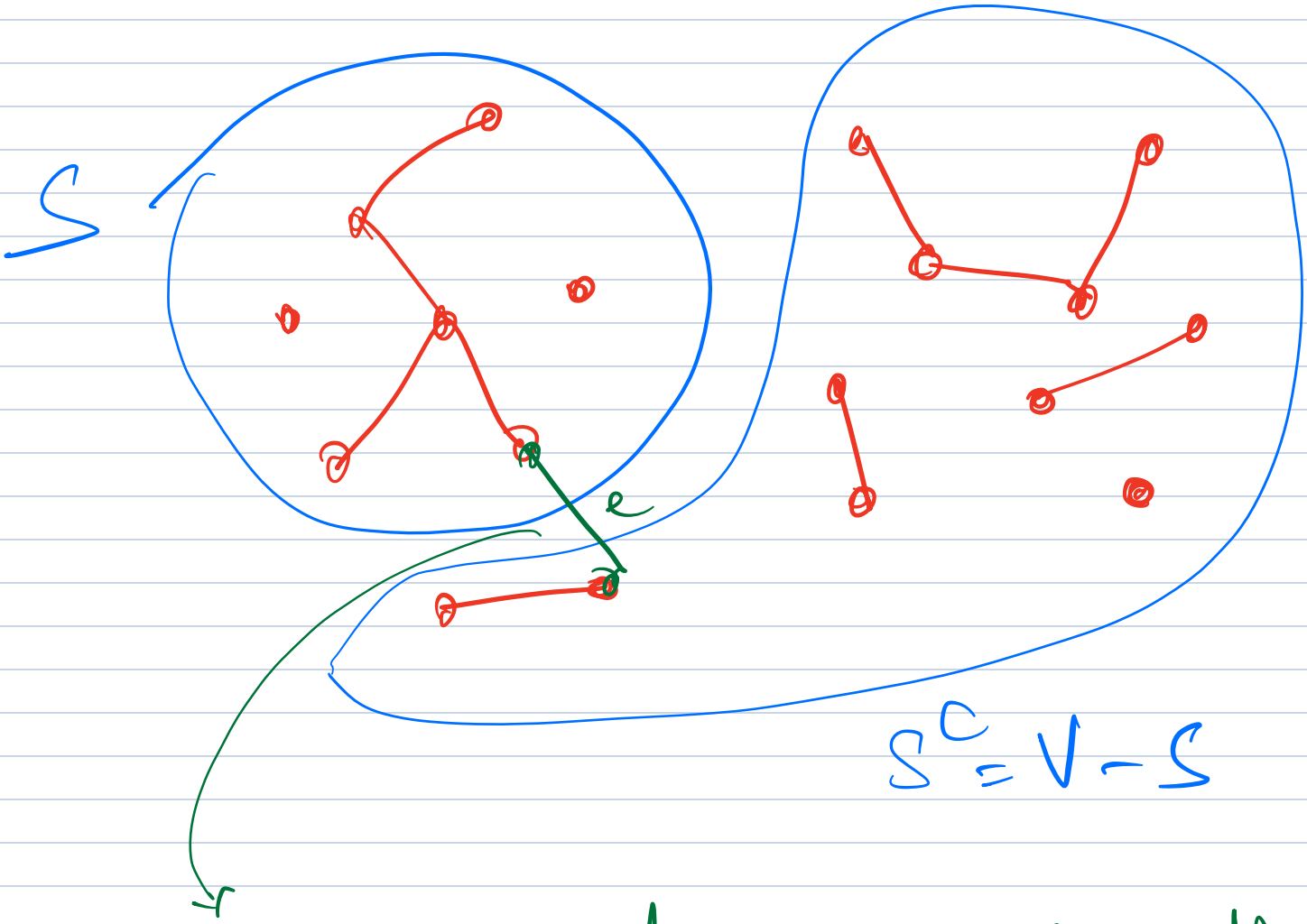
Cut property.

→ Suppose $A \subseteq E$ can be extended to an MST.

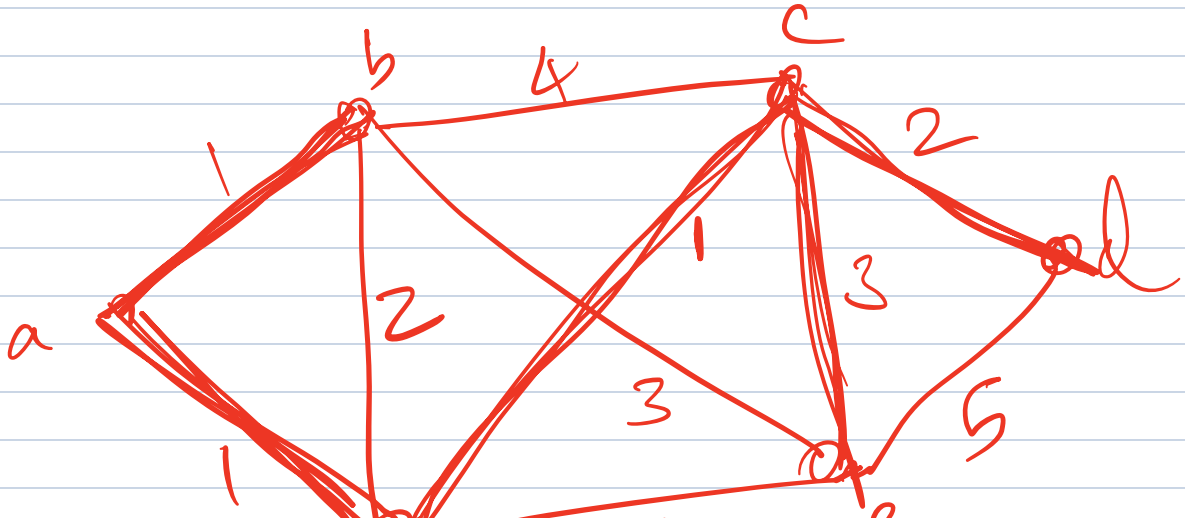
→ Let $S \subseteq V$ such that no edge in A has endpoints in both S and S^c $(V - S)$

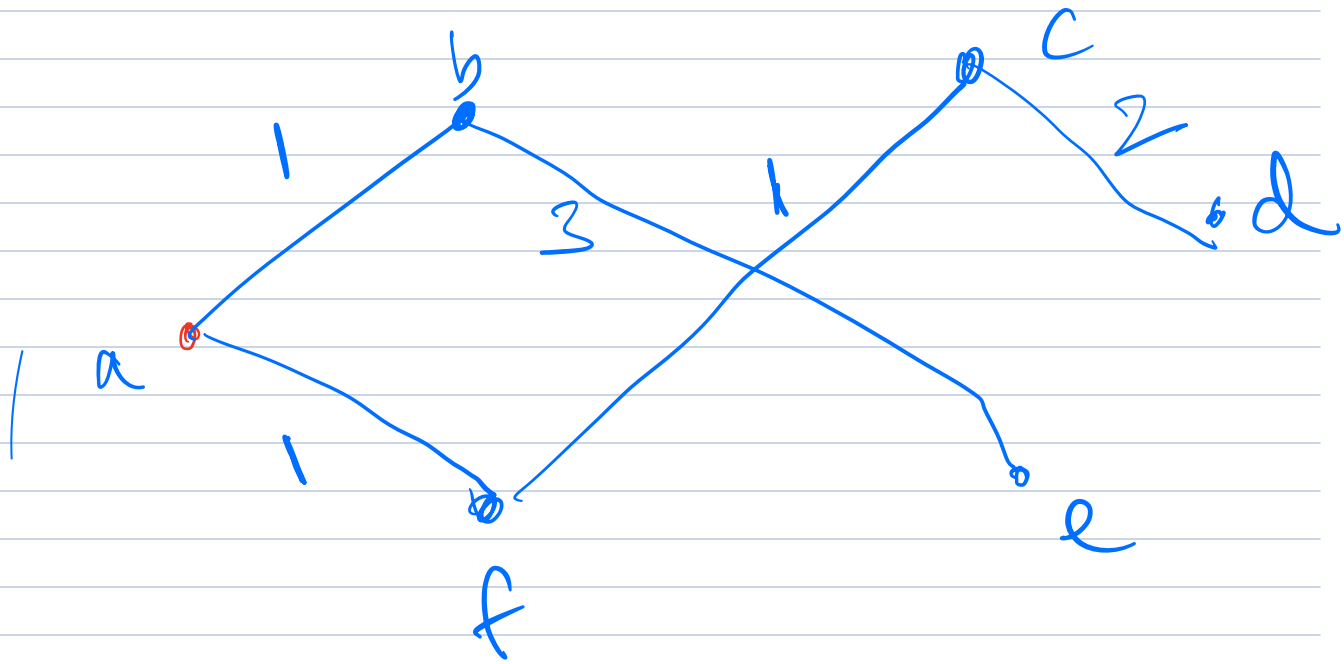
→ Then consider all the edges with one end point each in S and S^c. If e is such an edge with lowest weight, then e can be safely added

to A.



e is lowest cost edge that crosses the cut.




b
c
1
2
3
1
d
a
1
e
f